CVE-2020-25659 "fix" confirmed as insufficient #9785

Closed

tomato42 opened on Oct 26, 2023 · edited by tomato42 Edits •••

Different comments around CVE-2020-25659 (like in #6167 and #5507) state that the fix is incomplete and documentation (https://cryptography.io/en/latest/limitations/#rsa-pkcs1-v1-5-constant-time-decryption) specifies that PKCS#1 v1.5 encryption padding is vulnerable to side-channel attacks.

I'm filing this issue with the evidence that this is a confirmed fact, not just an assertion based on code analysis.

While I've executed it with python3-cryptography-3.2.1-6.el8.x86_64, python36-3.6.8-38.module+el8.5.0+12207+5c5719bc.x86_64, and openssl-1.1.1k-10.el8_9 both the cryptography package has the CVE-2020-25659 fix and openssl has the CVE-2022-4304 fix.

The test was executed with 2048 bit RSA, with 100k repeats per probe, in a VM on a fairly noisy/busy 2.3GHz Icelake Xeon system.

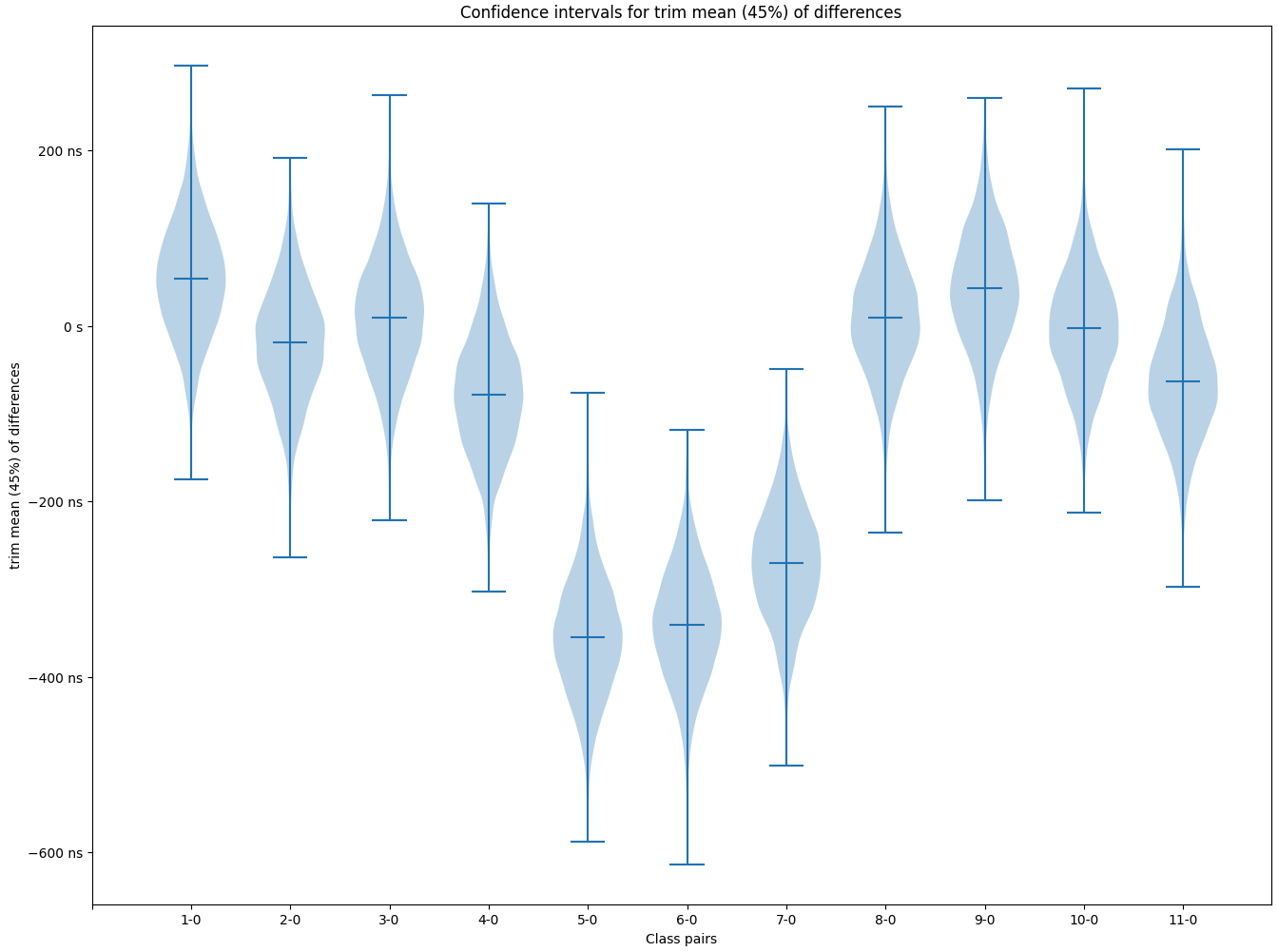
The measured side channel is about 400ns, so it should be fairly easy to exploit even over remote network connections.

analysis.py summary:

```
tlsfuzzer analyse.py version 5 analysis
Sign test mean p-value: 0.3447, median p-value: 0.1886, min p-value: 2.459e-13
Friedman test (chisquare approximation) for all samples
p-value: 8.947806111025913e-27
Worst pair: 1(no_header_with_payload_48), 6(valid_48)
Mean of differences: -5.35165e-07s, 95% CI: -1.51931e-06s, 4.859572e-07s (±1.003e-06s)
Median of differences: -4.50001e-07s, 95% CI: -5.85012e-07s, -3.309875e-07s (±1.270e-07s)
Trimmed mean (5%) of differences: -5.91658e-07s, 95% CI: -9.38480e-07s, -2.549461e-07s
(±3.418e-07s)
Trimmed mean (25%) of differences: -5.80887e-07s, 95% CI: -7.53559e-07s, -4.100913e-07s
(±1.717e-07s)
Trimmed mean (45%) of differences: -4.75457e-07s, 95% CI: -6.04405e-07s, -3.463844e-07s
(±1.290e-07s)
Trimean of differences: -6.01688e-07s, 95% CI: -8.09003e-07s, -3.924847e-07s (±2.083e-
07s)
```

Results for comparisons between individual samples are in the report.csv

Graphical representation of the confidence intervals of the difference between different classes to class 0:



mapping numbers to classes is in legend.csv
explanation of the ciphertexts generated is in the step2.py script.

You can reproduce this results using the script and instructions in https://github.com/tomato42/marvin-toolkit/tree/master/example/pyca-cryptography

1

reaperhulk on Oct 26, 2023 Member

Thanks for the analysis **@tomato42** 😄 Are your OpenSSL changes to expose useful constant time APIs available in the OpenSSL 3.2.0 beta?

tiran on Oct 26, 2023 Contributor

Yes, Hubert's `rsa_pkcs1_implicit_rejection` feature is in OpenSSL 3.2. Even better, it is **enabled** by default.

OpenSSL 3.0 in CentOS 9 Stream and RHEL 9.3 have a backport of the feature. (And yes, the backport broke `test_decrypt_invalid_decrypt`).

1

tomato42 on Oct 27, 2023 Author

Yes, it has been merged to 3.2.0 branch before first alpha was released: openssl/openssl#13817

reaperhulk on Oct 27, 2023 Member

We landed a skip for that test when handling the implicit rejection in `7e33b0e`

We'll undoubtedly start shipping OpenSSL 3.2.0 in our wheels when it is final, and at least one distribution has already backported -- is there anything we should change in our documentation here?

tomato42 on Oct 27, 2023 Author

Maybe link from documentation also to this issue, so that it's clear that the statements in the documentation are backed by real data?

Technically, it could list that the API will be secure if used together with OpenSSL that implements implicit rejection, but then *people really should stop using PKCS#1v1.5* so I'd rather not do that.

If you decide to fix it, I think a small rephrasing in that section would be good:

```
For this reason we recommend not implementing online protocols that use
RSA PKCS1 v1.5 decryption with cryptography –
independent of this limitation, such protocols generally have poor
security properties due to their lack of forward security.
```

The problem is not online protocols, it's any kind of interface which the attacker can measure the timing of. Online *contexts,* as stated in first sentence, would be better. But then there are situations in which seemingly offline systems can be attacked too:
https://arstechnica.com/information-technology/2023/06/hackers-can-steal-cryptographic-keys-by-video-recording-connected-power-leds-60-feet-away/
or
https://arstechnica.com/information-technology/2013/12/new-attack-steals-e-mail-decryption-keys-by-capturing-computer-sounds/
(the same side-channels can be used to measure processing much more precisely, or in situations where the decryptions are performed in batch operations, e.g. an email client that decrypts messages right after downloading them)

tomato42 on Oct 27, 2023 · edited by tomato42 Edits Author

ah, I also wonder about the status of CVE-2020-25659 itself, it is marked as "fixed", but that's not a good reflection of reality... I think a new one my be a right approach...

tomato42 mentioned this on Oct 27, 2023

Mark use of `PKCS1v15` for encryption and decryption a vulnerability PyCQA/bandit#1071

reaperhulk mentioned this on Oct 28, 2023

OpenSSL 3.2 features to expose #9795

alex added this to the Forty Second Release milestone on Dec 7, 2023

tomato42 on Dec 15, 2023 — Author

A CVE-2023-50782 has been assigned to track this issue.

kelvin-j-li on Jan 8, 2024

i saw this from cryptography milestones -- https://github.com/pyca/cryptography/milestone/44
Does any one know when release 42 will be available?

Thanks!

alex on Jan 8, 2024 via email — Member

Probably in 2-3 weeks.

...

1

kelvin-j-li on Jan 9, 2024 via email

[like] Kelvin Li reacted to your message:

...

alex on Jan 23, 2024 — Member

We're about to perform release 42.0, which will resolve this via upgrading our OpenSSL to 3.2.0.

Users who manually link against older OpenSSL's will still be impacted, but at some level that should be considered an issue with their OpenSSL.

alex closed this as completed on Jan 23, 2024

kelvin-j-li on Jan 23, 2024 · edited by kelvin-j-li — Edits

hi @**alex** ,
Thanks for your update!

I can see that cryptography-42.0.0 is now available in pip3 installation.

> We're about to perform release 42.0, which will resolve this via upgrading our OpenSSL to 3.2.0.

Which openssl library do you refer here? Is it pyOpenSSL? The latest for pyOpenSSL is still at version 23.3.0, and it is **incompatible** with cryptography-**42.0.0**:

```
ERROR: pip's dependency resolver does not currently take into account all the packages
that are installed. This behaviour is the source of the following dependency conflicts.
pyopenssl 23.3.0 requires cryptography<42,>=41.0.5, but you have cryptography 42.0.0
which is incompatible.
```

Is it an issue with https://github.com/pyca/pyopenssl?

Best regards

alex on Jan 23, 2024 Member

Ah yes, we need to issue a pyopenssl release. We'll do that shortly.

1

kelvin-j-li on Jan 23, 2024

hi **@alex**,
Wow, that's super quick! I saw your commit in pyopenssl just now:

pyca/pyopenssl@ `7f3e4f9`

Thanks so much!

kelvin-j-li on Jan 24, 2024

hi **@alex** ,
Do you know when CVE will be updated given that cryptography-42.0.0 is already released?

https://security.snyk.io/vuln/SNYK-PYTHON-CRYPTOGRAPHY-6126975 # it still has the statement -- There is no fixed version for cryptography.
https://www.cve.org/CVERecord?id=CVE-2023-50782

Is it upto Snyk to process / verify the fix?

Thanks.

reaperhulk on Jan 24, 2024 Member

Yes, snyk needs to update their database. There is no additional work on our end that we are aware of.

1 2

charn mentioned this on Jan 29, 2024

PUIS-78: Upgrade Python 3.7 -> 3.9, Django 2.2 -> 3.2 City-of-Helsinki/haravajarjestelma#134

tiran on Jan 31, 2024 Contributor

FWIW, the fix is not in PyCA cryptography but in OpenSSL 3.2.0. If you use the cryptography binary wheels, then you have to update to 42.0.0.

If you use downstream Linux or BSD packages from your vendor, then the version of PyCA cryptography does **not** matter. Your vendor must either ship OpenSSL 3.2.0 or have a backport of the 3.2.0 fix. Supported versions of Fedora, RHEL 8/9, and CentOS stream c8s/c9s have a fix for the timing problem for a while now.

There is an easy way to detect whether a build is affected by the timing vulnerability. If the following code raises an exception, then you are vulnerable. If it returns random junk, then your OpenSSL has `EVP_PKEY_CTRL_RSA_IMPLICIT_REJECTION` enabled and you are fine.

```
>>> from cryptography.hazmat.primitives.asymmetric import padding, rsa
>>> rsa.generate_private_key(65537, 2048).decrypt(b"\x00" * 256, padding.PKCS1v15())
```

11

xiaoge1001 on Feb 23, 2024 · edited by xiaoge1001 Edits

I have a question.
This problem occurs because cryptography invokes the OpenSSL API, and the decryption time of the corresponding OpenSSL API is not constant-time logic (prior to OpenSSL version3.2.0). Why is this CVE not assigned to OpenSSL (a new CVE number)? After all, directly invoking the OpenSSL interface in other ways also leak time information.

xiaoge1001 on Feb 23, 2024

CVE-2020-25659 has a score of 5.9, and it's AC is high. Similar CVE-2022-4304 also has a score of 5.9 and AC is high. Why does CVE-2023-50782 score 7.5 and it's AC decreases?

tomato42 on Feb 23, 2024 Author

> This problem occurs because cryptography invokes the OpenSSL API, and the decryption time of the corresponding OpenSSL API is not constant-time logic.

No, since the fix to CVE-2022-4304 OpenSSL's API will decrypt RSA ciphertexts in constant-time.

The problem is that before OpenSSL 3.2.0 (or before inclusion of implicit rejection) OpenSSL will return errors in case padding check fails. It still returns errors in the same amount of time as a result from decryption, so the API is side-channel safe. The problem is that pyca/cryptography changes that error into an exception, which causes different code path to be taken, and thus make execution take different amount of time. It's pyca/caryptography that introduces the side-channel leakage.

In 3.2.0 (or with implicit rejection) OpenSSL doesn't return errors, it returns a deterministically random message in case padding check fails. Since there is no error[1], pyca/crypptography doesn't raise an exception and thus doesn't create a side-channel leakage.

1 - more precisely, implicit rejection decouples the error being returned from the PKCS#1v1.5-compliance of the plaintext, which makes the API return uncorrelated with the first byte of the plaintext being a zero byte. See the Marvin paper for details.

Yes, both the score and description for CVE-2023-50782 is incorrect, it's not about TLS, it's about RSA PKCS#1v1.5 decryption. And the CVSS should be the same as for CVE-2020-25659. I've send an email to Red Hat Product Security to fix that, but I guess it got stuck in queue or something...

2

xiaoge1001 on Feb 24, 2024 •••

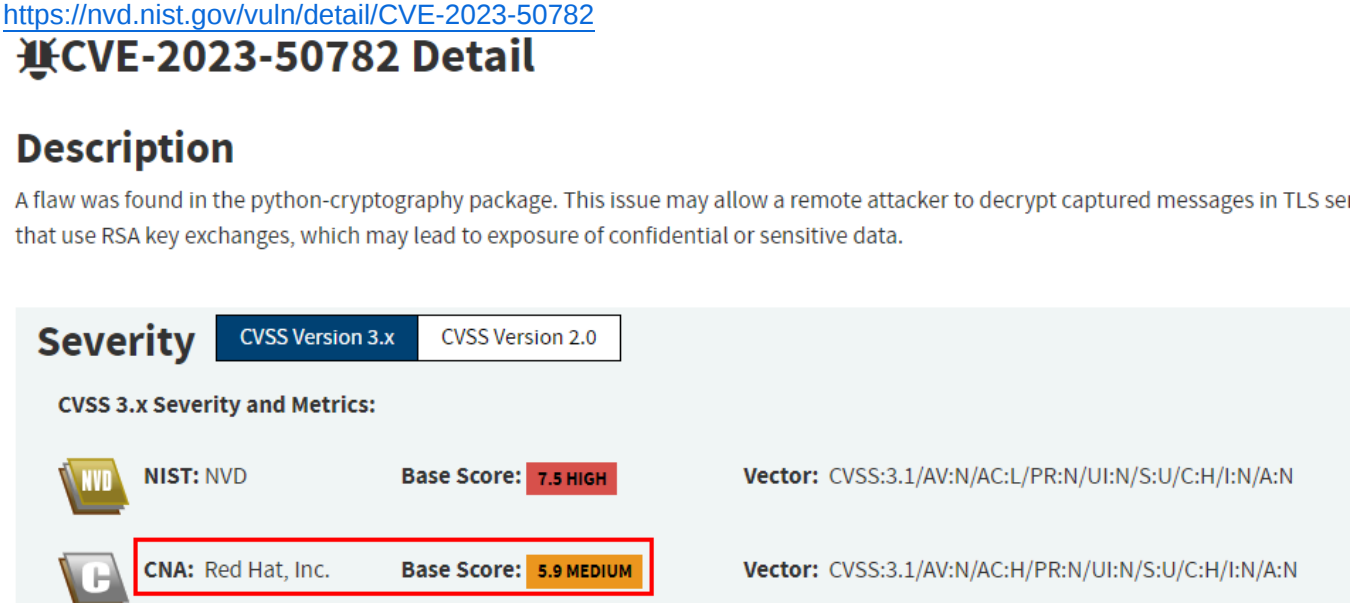
> This problem occurs because cryptography invokes the OpenSSL API, and the decryption time of the corresponding OpenSSL API is not constant-time logic.
>
> No, since the fix to CVE-2022-4304 OpenSSL's API will decrypt RSA ciphertexts in constant-time.
>
> The problem is that before OpenSSL 3.2.0 (or before inclusion of implicit rejection) OpenSSL will return errors in case padding check fails. It still returns errors in the same amount of time as a result from decryption, so the API is side-channel safe. The problem is that pyca/cryptography changes that error into an exception, which causes different code path to be taken, and thus make execution take different amount of time. It's pyca/caryptography that introduces the side-channel leakage.
>
> In 3.2.0 (or with implicit rejection) OpenSSL doesn't return errors, it returns a deterministically random message in case padding check fails. Since there is no error[1], pyca/crypptography doesn't raise an exception and thus doesn't create a side-channel leakage.
>
> 1 - more precisely, implicit rejection decouples the error being returned from the PKCS#1v1.5-compliance of the plaintext, which makes the API return uncorrelated with the first byte of the plaintext being a zero byte. See the Marvin paper for details.
>
> Yes, both the score and description for CVE-2023-50782 is incorrect, it's not about TLS, it's about RSA PKCS#1v1.5 decryption. And the CVSS should be the same as for CVE-2020-25659. I've send an email to Red Hat Product Security to fix that, but I guess it got stuck in queue or something...

Ok, I got the general idea. Thanks very much for your reply.

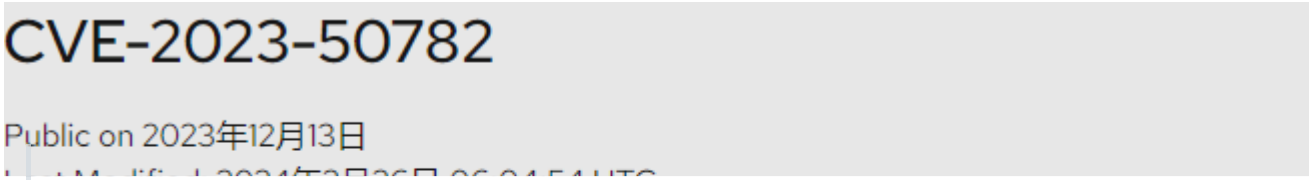
xiaoge1001 on Feb 26, 2024 · edited by xiaoge1001 Edits ▾ •••

I find that Red Hat modified the score for CVE-2023-50782 to 7.5 .

https://nvd.nist.gov/vuln/detail/CVE-2023-50782



https://access.redhat.com/security/cve/cve-2023-50782



xiaoge1001 mentioned this on Feb 29, 2024

ask the question about mitigating or fixing Bleichenbacher attacks on RSA decryption #10506

mark2093 mentioned this on May 6, 2024

Update cryptography dependency to 42.0.0 or higher paramiko/paramiko#2390

github-actions locked as resolved and limited conversation to collaborators on May 27, 2024

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

Forty Second Release

Closed on Jan 23, 2024, 100% complete

Relationships

None yet

Development

Code with agent mode

No branches or pull requests

Participants

 +1